

Oznacz jako wykonane

Komunikacja PP

Ćwiczenia poniżej proszę traktować jako propozycję zapoznania się z materiałem niezbędnym do realizacji zadania. Oczywiście jeśli ktoś woli przejść przez dokumentację MPI, może te ćwiczenia pominąć.

Ćwiczenie 1

- Zapoznaj się ze sposobem działania i różnicami poszczególnych typów komunikacji P2P dostępnych w MPI. Zaimplementuj przesłanie (tam i z powrotem) komunikatu między dwoma węzłami wykorzystując poniższe typy komunikacji PP.
 - Standard send `MPI_Send`
 - Synchronous send `MPI_Ssend`
 - Buffered send `MPI_Bsend`
 - Ready send `MPI_Rsend`
 - Non-blocking send `MPI_Isend` + `MPI_Wait`, `MPI_Test`
 - Odbieranie danych `MPI_Recv`, `MPI_Irecv`
- **Jak działa komunikacja standardowa?**
- <https://kompendium.plgrid.pl/technologie/mpi/>
- <https://www.mcs.anl.gov/research/projects/mpi/sendmode.html>
- Uwaga na `MPI_Rsend`: https://rookiehpc.org/mpi/docs/mpi_rsend/index.html

Ćwiczenie 2

- Zaimplementuj aplikację, gdzie 2 węzły wymieniają się komunikatami (ping-pong).
- Przetestuj jak zmienia się działanie aplikacji w dwóch przypadkach komunikacji: podanej przez prowadzącego na zajęciach i wybranej przez siebie w zależności od wielkości komunikatu.
- Zmierz opóźnienie (przyda się funkcja `MPI_Wtime`) - za opóźnienie przyjmujemy czas przesłania najmniejszego komunikatu - daje nam to dobre oszacowanie opóźnienia związanego z całym stosem programowym.
- **W jaki sposób należy dokonać pomiaru?** Należy sprawdzić, ile trwa przesłanie komunikatu (tam i z powrotem - czyli w rzeczywistości dwa komunikaty) N razy (gdzie $N > 1000$) i następnie podzielić otrzymany czas by otrzymać czas przesłania pojedynczego komunikatu.
- **Do czego służy `MPI_Barrier`?** Wykorzystaj barierę do ustalenia, że wszystkie procesy komunikujące się już wystartowały. Po komunikacji P2P nie musisz korzystać z bariery - tutaj komunikacja jest synchronizacją.

Ćwiczenie 3

- Zaimplementuj aplikację, w której każdy węzeł przesyła do kolejnego otrzymaną daną, a ostatni do pierwszego - **token ring**.
- Przetestuj jak zmienia się działanie aplikacji w dwóch przypadkach komunikacji: zwykłej i synchronicznej.
- Jak poprzednio dokonaj pomiarów.

Zadanie domowe

Celem zadania jest zmierzenie wartości opóźnienia i charakterystyki przepustowości połączeń w klastrze.

- Należy przetestować dwa różne typy komunikacji P2P w MPI.
- Aby dostać punkty za aktywność na zajęciach, wystarczy przetestować jeden typ komunikacji, podany przez prowadzącego (upewnić się, że działa).
- Należy dokonać pomiarów (patrz ćw. 2):
 - przepustowości [Mbit/s] od długości komunikatów [B]: wykres,
 - opóźnienia [ms] przesyłania krótkiego komunikatu: wartość (opóźnienie definiujemy tutaj jako szczególny przypadek pomiaru przepustowości dla bardzo małego komunikatu, np. 1B).
- Implementacja w C lub Python.
- Sugestia: Do rysowania wykresów można użyć [Gnuplot](#), [pyplot](#), [R/ggplot2](#).
- **Zwrócić uwagę czy podczas testu maszyna jest obciążona - ktoś inny może puszczać w tym samym czasie testy, co zaburza wyniki.**
- Proszę przeprowadzić testy w następujących konfiguracjach:
 - Komunikacja na 1 nodzie - należy wziąć dwa procesory z jednej maszyny, spośród [vnode-01, ..., vnode-04].

- Komunikacja między 2 nodami - należy wziąć po jednym procesorze z dwóch maszyn, spośród [vnode-05, ..., vnode-12].

- **Wynikiem zadania jest pierwsza część sprawozdania z MPI.**

Elementy konieczne do sprawozdania:

- Kody programów (jako załącznik, nie w PDF).
- Dane pomiarowe (załącznik), krótki komentarz odnośnie ich pozyskania (PDF).
- Opis strategii wykonania pomiarów.
- Wykresy na podstawie wyliczeń z danych pomiarowych - tytuł, opisane osie, jednostki, serie (jeżeli występują). Niezależnie od tego, czy wygenerowane są osobne wykresy czy serie na jednym wykresie, powinno być jasno widoczne, z jakiej konfiguracji uruchomienia pochodzą dane (tytuł wykresu albo opis serii).
- Opisy i wnioski do wykresów.

Ta część sprawozdania (w postaci PDF) powinna zajmować około 1-2 stronę (i być napisana w LaTeX, np. Overleaf, TeX Live).

Ostatnia modyfikacja: piątek, 13 marca 2026, 20:16

